ChatGPT - OPENCLAW

https://github.com/hkaanturgut want to build a website like this on github.com

If you want to build a GitHub profile/portfolio website like the one from **GitHub user hkaanturgut**, you're essentially building a **developer portfolio + profile README + optionally a full static site**.

Let's break this down into **what you're actually seeing vs how to replicate it** 👇

🧠 What that profile likely includes

Most modern GitHub profiles combine:

1. Profile README (main visual page)

- Custom banners / intro text
- Skills badges (DevOps, AI, Cloud)
- Stats widgets
- Project highlights

2. Pinned repositories

- Clean, curated repos
- Each repo = mini portfolio project

3. Optional GitHub Pages website

- Full portfolio site (React / static site)
- Hosted via GitHub Pages

🚀 Step 1 — Create your GitHub Profile README

This is the **core piece** (fastest win).

Create a special repo:

```
repo name = your-username
```

Example:

```
DattaramMiruke/DattaramMiruke
```

Then add:

```Markdown
# 👋 Hi, I'm Dattaram Miruke

🚀 DevOps | AI Infrastructure | Platform Engineering

- 🔧 AWS | Kubernetes | Terraform | GitOps
- 🧠 LLM | RAG | Agent Systems | Observability
- 📈 Building AI + DevOps platforms

---

## 🔥 Featured Projects

- 🚀 DevOps Agent System (Terraform + AI)
- 🧠 RAG Evaluation Platform (AEGIS-inspired)
- 📊 Observability Stack (Prometheus + Grafana + OpenTelemetry)

---

## 📊 GitHub Stats

![Stats](https://github-readme-stats.vercel.app/api?username=YOUR_USERNAME)

---

## 🌐 Connect
- LinkedIn
- Portfolio (coming soon)
```

🎨 Step 2 — Add Visual Enhancements

These are what make profiles like hkaanturgut stand out.

🔹 Badges

Use:

- shields.io

Example:

```Markdown
![AWS](https://img.shields.io/badge/AWS-Cloud-orange)
![Kubernetes](https://img.shields.io/badge/K8s-Orchestration-blue)
```

🔹 GitHub Stats

- github-readme-stats
- streak stats

🔹 Animated headers

- GIF banners (optional)
- SVG typing animations

🌐 Step 3 — Build a Full Website (GitHub Pages)

This is where you go **beyond profile → real portfolio**

Option A (FAST): Static HTML site

Create repo:

```
portfolio-site
```

Add:

```
index.html
style.css
```

Enable:
Settings → Pages → Deploy from branch

Option B (BEST): React + Modern UI

Use:

- React
- Next.js

```Bash
npx create-next-app portfolio
cd portfolio
npm run build
```

Deploy via:

- GitHub Pages OR
- Vercel (better)

Option C (DEVOPS STYLE — fits YOU best)

Build something like:

🔧 Stack:

- Frontend: Next.js
- Backend: FastAPI
- Infra: Terraform
- CI/CD: GitHub Actions
- Hosting: AWS S3 + CloudFront

This becomes a **portfolio project itself**

🧠 What YOU should build (based on your background)

Don't just copy — **differentiate**

Your version should include:

🔴 1. DevOps + AI Portfolio Dashboard

- Projects auto-fetched from GitHub API
- Architecture diagrams (your style)
- Terraform previews

🔴 2. Live System Visualizations

You already mentioned:
👉 visdoc

Embed:

`http://37.27.97.75:8080/visdoc`

🔴 3. Agent-based demo

- DevOps Agent (plan infra)
- Evaluation Agent
- Architecture scorer

🔴 4. Interactive Resume

- LLM-powered Q&A about your experience
- "Ask about my projects"

📁 Suggested Repo Structure

```
github-profile/
portfolio-site/
devops-agent-system/
rag-eval-platform/
infra-terraform-modules/
```

⚡ Fastest Path (no overthinking)

Day 1:

- Create profile README
- Add badges + stats

Day 2:

- Create simple portfolio site (HTML or Next.js)

Day 3–7:

- Add 2–3 **real projects**
  (this matters more than design)

🧠 Reality check (important)

Most people:

- Overdesign
- Underbuild

What actually matters:
✔️ 3 strong projects
✔️ Clean GitHub profile
✔️ Working demos

🚀 If you want next step

I can build for you:

🔥 A complete GitHub-ready setup:

- Profile README (optimized for DevOps/AI roles)
- Portfolio website (Next.js + deployed)
- 3 flagship projects:
  - DevOps Agent
  - RAG Eval System
  - Infra Generator

Just say:
👉 *"build full portfolio system"*

and I'll generate everything end-to-end.

---

here I can link my prohjects portfolio as well

Yes — and you absolutely **should**. That's actually the whole point of a strong GitHub presence.

But don't just "link projects." Do it in a way that **tells a story + proves capability** (especially for the kind of DevOps + AI roles you're targeting).

🔗 Where you can link your projects

1. Profile README (Primary Hub)

This is your **landing page inside GitHub**

Structure it like this:

```Markdown
# 🚀 Portfolio

## 🔴 Flagship Projects

### 🧠 DevOps Agent System
➡ https://github.com/your-username/devops-agent
✔ Generates Terraform + CI/CD pipelines
✔ Architecture scoring engine
✔ Multi-cloud support

---

### 📊 RAG Evaluation Platform (AEGIS-style)
➡ https://github.com/your-username/rag-eval
✔ LLM evaluation (RAGAS, DeepEval)
✔ Observability + scoring
✔ Trust & governance layer

---

### ⚙ Infra Generator (Terraform + AI)
➡ https://github.com/your-username/infra-gen
✔ Converts intent → infrastructure
✔ AWS + Kubernetes support
✔ CI/CD integrated

---
```

👉 This is what recruiters will scan in **10–15 seconds**

---

2. Pin Repositories (Critical)

At the top of your profile:

- Pin **6 repos max**
- Choose:
  - 2 AI projects
  - 2 DevOps projects
  - 1 Infra project
  - 1 System design / visualization

👉 These act as your **visual portfolio cards**

3. GitHub Pages / Portfolio Website

Now connect everything:

Example sections:

```
Home
Projects
Architecture Diagrams
Live Demos
Resume
Contact
```

Each project should include:

- GitHub link
- Live demo (if possible)
- Architecture diagram (your strength)
- Tech stack
- Problem → Solution → Impact

🧠 What makes your portfolio strong (important)

Most people do:
❌ "Here's my repo"

You should do:
✅ "Here's the system I built + why it matters"

🔥 Ideal Project Card (use this format)

```Markdown
## 🧠 DevOps Agent System

Problem:
Manual infra design is slow and inconsistent.

Solution:
Built an AI agent that converts intent → Terraform + CI/CD pipelines.

Architecture:
(User → LLM → Planner → Terraform Generator → GitHub Actions → AWS)

Tech:
Terraform | AWS | LangChain | FastAPI | GitHub Actions

👉 Repo: https://github.com/your-username/devops-agent
👉 Demo: https://your-site/demo
```

🔴 What YOU should emphasize (based on your background)

Don't make it generic. Highlight:

- Kubernetes internals
- Multi-cloud infra
- AI + RAG systems
- Observability + reliability
- Agent-based automation

⚡ Advanced (this is where you stand out)

1. Auto-sync projects to website

- Use GitHub API
- Show latest repos dynamically

---

2. Add live demos

Examples:

- Terraform plan preview
- Architecture visualizer (visdoc)
- RAG query demo

---

3. Add “Ask Me Anything” AI

- LLM trained on your projects
- “Ask about my experience”

🧠 Brutal truth

Recruiters don’t care about:

- Fancy UI
- Long descriptions

They care about:
✔️ Can you build systems
✔️ Can you explain architecture
✔️ Can you deploy

🚀 Recommended setup for YOU

Profile README → acts as:

👉 "Executive summary"

GitHub repos → act as:

👉 "Proof"

Portfolio site → acts as:

👉 "Presentation layer"

If you want

I can generate:

✅ Your **complete GitHub README (tailored to your roles)**
✅ Your **project descriptions (high-impact, recruiter-friendly)**
✅ Your **portfolio website structure + code (Next.js + DevOps)**

Just say:
👉 *"build my full GitHub portfolio"*

and I'll assemble it end-to-end.

---

show me how to create a .md to display on github

Creating a `.md` (Markdown) file for GitHub is simple—but doing it *well* is what makes your profile stand out.

Let's walk through it step-by-step 👇

🧱 1. What is a `.md` file?

A `.md` file = **Markdown file**
It's just text + formatting that renders nicely on GitHub.

Used for:

- `README.md` → shown on repo homepage
- Profile README → shown on your profile
- Docs / guides

⚙️ 2. Create a `.md` file (3 ways)

Option A — Directly on GitHub (fastest)

1. Go to your repo
2. Click **"Add file"** → **"Create new file"**
3. Name it:

```Bash
README.md
```

4. Add content
5. Click **Commit**

Option B — Locally (recommended)

```Bash
touch README.md
```

Open in editor:

- VS Code / Notepad++

Then push:

```Bash
git add README.md
git commit -m "Add README"
git push
```

Option C — For Profile README (special case)

Create repo named exactly:

```Bash
your-username
```

Example:

```Bash
DattaramMiruke/DattaramMiruke
```

Then add:

```Bash
README.md
```

👉 This shows on your profile homepage

✍️ 3. Basic Markdown Syntax

Headings

Markdown
```
# Title
## Section
### Subsection
```

Text formatting

Markdown
```
**bold**
*italic*
~~strikethrough~~
```

Lists

Markdown
```
- Item 1
- Item 2

1. Step one
2. Step two
```

Links

Markdown
```
[My Repo](https://github.com/your-username/repo)
```

Code blocks

Markdown
```
```bash
terraform apply
```
```

Inline code

Markdown
```
Use `kubectl apply`
```

🎯 4. Example: Simple Portfolio README

Copy-paste this and modify:

Markdown
```
# 👋 Hi, I'm Dattaram Miruke

🚀 DevOps | AI Infrastructure | Platform Engineering

---

## 🔧 Skills

- AWS | Kubernetes | Terraform
- CI/CD | GitHub Actions | Jenkins
- LLM | RAG | Agent Systems

---

## 🔴 Projects

### 🧠 DevOps Agent System
AI agent that generates Terraform + CI/CD pipelines
👉 https://github.com/your-username/devops-agent
```

```
---

### 📊 RAG Evaluation Platform
LLM evaluation + observability + scoring
👉 https://github.com/your-username/rag-eval

---

## 📊 GitHub Stats

![Stats](https://github-readme-stats.vercel.app/api?username=YOUR_USERNAME)

---

## 🌐 Contact

- LinkedIn: your-link
- Email: your-email
```

🎨 5. Add Visual Enhancements (optional but powerful)

Badges

```Markdown
![AWS](https://img.shields.io/badge/AWS-Cloud-orange)
![Kubernetes](https://img.shields.io/badge/K8s-Orchestration-blue)
```

Images

```Markdown
![Architecture](./images/architecture.png)
```

GIF / Banner

```Markdown
![Banner](https://your-image-link.gif)
```

🧠 6. Pro Tips (this matters more than syntax)

❌ Don't do:

- Huge walls of text
- Random project dumps

✅ Do:

- Highlight **3–6 strong projects**
- Use **clear sections**
- Add **architecture + impact**

🔥 7. Advanced (for you)

Given your background, your `.md` should include:

- Architecture diagrams (ASCII or images)
- Terraform snippets
- System design explanation
- Links to live demos (visdoc etc.)

⚡ Minimal Template (clean & effective)

```Markdown
# 👋 Dattaram Miruke

## 🚀 What I Do
DevOps + AI Infrastructure | Kubernetes | AWS | Terraform

## 🔥 Projects
- DevOps Agent → link
- RAG Eval System → link
- Infra Generator → link

## ⚙ Stack
AWS | Kubernetes | Terraform | Python | Go

## 📫 Contact
LinkedIn | Email
```

🚀 If you want next level

I can generate:

- Your **full production-grade README.md**
- With **architecture diagrams + badges + stats**
- Tailored for **DevOps / AI roles**

Just say:
👉 *"generate my README.md"*

and I'll build it ready to paste.

DMIRUKE_RESUME_AI_SOL_ARCH.pdf
PDF

use this resume and create a .md file alon the lines of the https://github.com/hkaanturgut

Below is a **GitHub-ready `README.md`** tailored from your resume DMIRUKE_RESUME_AI_SOL_ARCH and styled along the lines of high-end profiles like hkaanturgut — but adapted to **your DevOps + AI Platform + Agentic Systems positioning**.

You can **copy-paste this directly into your GitHub profile repo (`DattaramMiruke/README.md`)**.

🚀 `README.md` (Your Portfolio Profile)

Markdown

```markdown
# 👋 Hi, I'm Dattaram Miruke

### 🧠 Principal AI Platform Architect | DevOps | Distributed Systems | Kubernetes | AWS

📍 Sacramento, California
📧 dattamiruke@gmail.com

---

## 🚀 About Me

I design and build **production-grade AI platforms and cloud-native systems** at scale.

- 🧠 AI Control Planes, Multi-Agent Systems, RAG Pipelines
- ⚙️ Kubernetes Platform Engineering, Distributed Systems
- ☁️ AWS Multi-Region Architectures, Event-Driven Systems
- 🔐 AI Governance, Observability, and Policy Enforcement

I specialize in turning **ambiguous problems → structured, scalable systems**.

---

## 🔴 Flagship Projects

### 🧠 AEGIS — AI Trust & Evaluation Framework
> Governance, evaluation, and safety layer for agentic AI systems

- LLM evaluation pipelines (RAGAS, DeepEval)
- Policy enforcement (OPA, RBAC)
- Observability + scoring engine
- Multi-agent validation workflows

👉 https://github.com/your-username/aegis

---

### ⚙️ DevOps Agent System (Infra-as-Code Generator)
> AI agent that converts intent → Terraform + CI/CD pipelines

- Multi-cloud infra generation (AWS/Kubernetes)
- Architecture scoring engine
- CI/CD automation (GitHub Actions)
- Plan preview + validation

👉 https://github.com/your-username/devops-agent

---

### 📊 RAG + Document Intelligence Platform
> End-to-end ingestion → embedding → retrieval → evaluation

- Semantic search pipelines (vector + metadata)
- PromptBuilder for structured prompt generation
- Multi-channel ingestion (docs, APIs, streams)
- Persistent AI memory (OpenBrain system)

👉 https://github.com/your-username/rag-platform

---

### 🧠 OpenBrain — Persistent AI Memory System
> Vector + metadata-based memory for context-aware agents

- Long-term memory for LLM systems
- Context enrichment pipelines
- Multi-agent memory sharing
- Retrieval optimization
```

```
👉 https://github.com/your-username/openbrain

---

## 🧮 Architecture Focus
```

User → API Gateway → LLM Control Plane → Agent Orchestrator
→ Tool Execution Layer → Memory (Vector + Metadata)
→ Observability + Evaluation → Policy Enforcement

```
---

## ⚙️ Tech Stack

### 🧠 AI / Agent Systems
- LLM Control Planes
- LangChain / LangGraph
- RAG Pipelines
- Multi-Agent Orchestration
- Prompt Engineering Frameworks

### ☁️ Cloud & Infrastructure
- AWS (EKS, Lambda, S3, RDS, Bedrock)
- Multi-Region HA Architectures
- Event-Driven Systems (SNS/SQS, Step Functions)

### ⚙️ DevOps / IaC
- Terraform, AWS CDK
- GitHub Actions, GitOps
- Kubernetes (EKS, CRDs, Controllers)

### 📊 Observability
- OpenTelemetry
- Prometheus, Grafana
- ELK, CloudWatch

### 🔐 Security & Governance
- OPA (Policy-as-Code)
- RBAC, Zero Trust
- AI Guardrails

---

## 📊 GitHub Stats

![GitHub Stats](https://github-readme-stats.vercel.app/api?username=YOUR_USERNAME&show_icons=true)

---

## 🔪 Experience Highlights

### 🧠 Principal AI Platform Architect — Inferloop
- Built multi-agent AI platform with Kubernetes + multi-LLM gateways
- Designed LLM control plane (routing, policy, cost optimization)
- Developed OpenBrain memory system (vector + metadata)
- Implemented AI evaluation + governance frameworks

---

### ☁️ AWS DevOps Architect (Consultant)
- Multi-region HA architectures on AWS
- Event-driven systems (Lambda, Step Functions, SNS/SQS)
- Terraform modules + CI/CD pipelines

---

### ⚙️ Senior DevOps Engineer — Amazon
- Built distributed event-driven systems
- IaC platforms using AWS CDK
- Observability frameworks (CloudWatch, DORA metrics)

---
```

```
### 🌐 Platform Engineering — Cisco / HP / Others
- Kubernetes CRDs + controllers for resilience
- Chaos engineering frameworks
- OpenStack / SDN / NFV systems
- Large-scale distributed system optimization

---

## 📚 Thought Leadership

- 📄 Agentic Assurance Index (Upcoming — arXiv)
- 📘 “Routing in the Internet” — CRC Press
- 🔪 Early security research (USENIX)

---

## 🔬 Additional Work

- HPC cluster for ML workloads (CUDA-based training)
- Drone-based video analytics (DNN/CNN models)
- AI anomaly detection systems

---

## 🌐 Live Systems / Visualization

- 🔗 VisDoc: http://37.27.97.75:8080/visdoc

---

## 📫 Connect

- LinkedIn: (add link)
- GitHub: https://github.com/your-username

---

## ⚡ Philosophy

> Build systems that are:
> - Observable
> - Scalable
> - Secure
> - Explainable

---
```

🧠 Why this works (important)

This is not just a resume dump. It:

✔️ Positions you as **AI Platform Architect (top tier)**
✔️ Shows **real systems (agents, RAG, infra)**
✔️ Aligns with **Staff / Principal roles**
✔️ Tells a **coherent story**

🔧 What YOU should do next (critical)

1. Replace:

```Bash
your-username
```

2. Create these repos (even if WIP):

- `aegis`
- `devops-agent`
- `rag-platform`
- `openbrain`

---

3. Pin them on your GitHub profile

🚀 If you want next step

I can now generate:

✅ Full **GitHub portfolio website (Next.js + DevOps)**
✅ Each project **complete repo structure + code**
✅ Terraform + CI/CD pipelines for deployment
✅ AI-powered "Ask my resume" system

Just say:
👉 *"build full portfolio system with repos"*

and I'll take this to production-level.

Sources

Can I view this on my desktop in browser?

Yes—you can view your `.md` file on your desktop in a browser, but not *directly* like an HTML page. You need one of these approaches 👇

✅ Option 1 — Open locally with a Markdown preview (fastest)

Easiest way (recommended):

Use **Visual Studio Code**

Steps:

1. Open your folder in VS Code
2. Open `README.md`
3. Press:

```bash
Ctrl + Shift + V
```

👉 This gives you a **rendered preview (like GitHub)**

✅ Option 2 — Use a browser-based Markdown viewer

Method:

- Install a browser extension (Chrome/Firefox)
- Open `.md` file directly

Example extensions:

- "Markdown Viewer"
- "Markdown Preview Plus"

Then:

```bash
Right click → Open with browser
```

✅ Option 3 — Convert `.md` → HTML (best for portfolio preview)

This is the closest to "real website view"

Install:

```bash
npm install -g marked
```

Convert:

```bash
marked README.md > index.html
```

Then open:

```bash
index.html (double click)
```

👉 Now it renders like a webpage

✅ Option 4 — Serve locally (DevOps-style, best for you)

Since you're building infra systems, do this:

```bash
python3 -m http.server 8080
```

Then open:

```bash
http://localhost:8080
```

👉 Works if you have `index.html`

🔥 Option 5 — View EXACTLY like GitHub (best)

Push to GitHub:

```bash
git add README.md
git commit -m "Add README"
git push
```

Then open:

```bash
https://github.com/your-username
```

👉 This is the **real rendering recruiters see**

🧠 Important Reality

- `.md` = not a website
- It needs a **renderer (GitHub / VS Code / HTML conversion)**

🚀 Best setup for YOU

Use this combo:

1. **VS Code preview** → fast iteration
2. **GitHub push** → final view
3. Later → convert into **portfolio site (Next.js)**

⚡ If you want next step

I can convert your README into:

✅ Full **portfolio website (HTML or Next.js)**
✅ With **sections, navigation, animations**
✅ Deployable on GitHub Pages

Just say:
👉 *"convert this to a website"*

---

will it be visible on github main page?

Yes — **it will be visible**, but *where* it shows depends on how you set it up on GitHub.

✅ Case 1 — Profile Main Page (what you want)

👉 This is the **top of your GitHub homepage**

Requirement:

Create a repo named **exactly your username**

Example:

`DattaramMiruke/DattaramMiruke`

Inside it:

`README.md`

Result:

➡️ When someone visits:

`https://github.com/DattaramMiruke`

👉 Your **`.md`** content shows **directly on your main profile page**

✅ Case 2 — Repository Page

If your **`.md`** is inside a normal repo:

Example:

`devops-agent/README.md`

Result:

➡️ It shows only here:

`https://github.com/DattaramMiruke/devops-agent`

👉 Not on your main profile

🧠 Visual Summary

```
GitHub Profile Page
│
├── Profile README (auto-shown here) ✅
│
├── Pinned Repositories
│     ├── Repo 1 (README inside)
│     ├── Repo 2
│     └── Repo 3
│
└── Contributions / Activity
```

🔥 What you should do (important)

1. Create profile repo

`DattaramMiruke/DattaramMiruke`

2. Add your generated README.md

---

3. Pin your best repos (6 max)

- AEGIS
- DevOps Agent
- RAG Platform
- OpenBrain
- Infra Generator
- Observability / VisDoc

⚡ Pro tip (this is what makes profiles like hkaanturgut strong)

They combine:
✔️ Profile README (story + overview)
✔️ Pinned repos (proof)
✔️ Clean structure

🚀 Final answer

👉 **Yes — your `.md` WILL be visible on your main GitHub page**
ONLY IF it is inside the repo named exactly your username.

If you want, I can:

- Optimize your README specifically for **recruiter scanning (10 sec impact)**
- Add badges, visuals, and layout improvements
- Structure pinned repos for maximum effect

Just say:
👉 *"optimize my GitHub profile for hiring"*

▢
optimize my GitHub profile for hiring"

Good—this is the part that actually changes outcomes.

Right now, your profile should be optimized for **one thing only**:
👉 *A recruiter spends ~10–15 seconds → decides "interview" or "skip"*

Let's make sure it lands on **"interview"**

🧠 1. Positioning (most important)

Don't present yourself as:
❌ "DevOps Engineer"
❌ "AI Engineer"

Present as:

👉 **Principal AI Platform & DevOps Architect**
(*this matches your resume reality* DMIRUKE_RESUME_AI_SOL_ARCH)

🚀 2. Your Optimized README (FINAL VERSION)

Replace your current README with this:

```Markdown
# 👋 Dattaram Miruke

### Principal AI Platform Architect | DevOps | Kubernetes | AWS | Distributed Systems

📍 Sacramento, CA
📧 dattamiruke@gmail.com

---

## 🚀 What I Build

I design and operate **production-grade AI + cloud platforms**:

- 🧠 AI Control Planes (multi-LLM routing, policy, cost optimization)
- ⚙️ Multi-Agent Systems (tool orchestration, reasoning workflows)
- ☁️ Distributed Cloud Infrastructure (AWS, Kubernetes, event-driven systems)
- 📊 Observability + Evaluation (LLM eval, telemetry, governance)

---

## 🔴 Flagship Systems (Proof of Work)

### 🧠 AEGIS — AI Trust & Evaluation Framework
AI governance, evaluation, and safety layer for agentic systems

- LLM evaluation (RAGAS, DeepEval)
- Policy enforcement (OPA, RBAC)
- Observability + scoring engine

👉 https://github.com/your-username/aegis

---

### ⚙️ DevOps Agent — Infra Generation System
AI agent that converts intent → Terraform + CI/CD pipelines

- Multi-cloud infra generation
- Architecture scoring engine
```

```
- GitHub Actions integration

👉 https://github.com/your-username/devops-agent

---

### 📊 RAG Platform — Document Intelligence System
End-to-end ingestion → embedding → retrieval → evaluation

- Semantic search pipelines
- PromptBuilder framework
- Persistent AI memory

👉 https://github.com/your-username/rag-platform

---

### 🧠 OpenBrain — AI Memory System
Persistent memory layer for context-aware agents

- Vector + metadata storage
- Multi-agent memory sharing
- Context enrichment pipelines

👉 https://github.com/your-username/openbrain

---

## 🧮 Architecture Snapshot
```

User → API → LLM Control Plane → Agent Orchestrator
→ Tool Execution → Memory (Vector + Metadata)
→ Observability → Policy Enforcement

```
---

## ⚙ Core Stack

**AI / Agents**
LLMs | RAG | LangChain | LangGraph | Multi-Agent Systems

**Cloud / Infra**
AWS (EKS, Lambda, S3, Bedrock) | Multi-region HA

**DevOps / IaC**
Terraform | GitHub Actions | Kubernetes

**Observability**
OpenTelemetry | Prometheus | Grafana | ELK

**Security**
OPA | RBAC | Zero Trust

---

## 🪚 Experience Highlights

- Built AI platform with multi-agent orchestration + LLM control plane
- Designed RAG pipelines + persistent memory systems
- Delivered multi-region AWS architectures
- Built Kubernetes-based CI/CD and resilience frameworks

---

## 🔬 Live Systems

- VisDoc: http://37.27.97.75:8080/visdoc

---

## 📊 GitHub Stats
```

```
![Stats](https://github-readme-stats.vercel.app/api?username=YOUR_USERNAME&show_icons=true)

---

## 📫 Connect

LinkedIn | GitHub

---

## ⚡ Focus

> Building scalable, observable, and governed AI systems.
```

🔥 3. Pin These Repositories (CRITICAL)

Go to your profile → “Customize pins”

Pin ONLY these (max 6):

MUST HAVE:

1. AEGIS
2. DevOps Agent
3. RAG Platform
4. OpenBrain

ADD:

5. Infra Generator (Terraform-heavy)
6. Observability / VisDoc

🧠 4. Each Repo Must Look Like This

Every repo should have:

```Markdown
# Project Name

## Problem
What real-world issue this solves

## Solution
What system you built

## Architecture
Diagram (ASCII or image)

## Tech Stack
Clear list

## How to Run
Commands

## Demo
(Optional)
```

👉 This is where most candidates fail

⚡ 5. Add Visual Signals (fast credibility)

Add badges at top of README:

```Markdown
![AWS](https://img.shields.io/badge/AWS-Cloud-orange)
![Kubernetes](https://img.shields.io/badge/K8s-Orchestration-blue)
![Terraform](https://img.shields.io/badge/Terraform-IaC-purple)
```

🚫 6. What to REMOVE

- Old/random repos
- Incomplete experiments
- Generic tutorials

👉 Archive them

🧠 7. What Recruiters Actually Look For

In order:

1. Title clarity (first 2 lines)
2. Real systems (not toy apps)
3. Architecture thinking
4. Stack alignment
5. Clean repos

🔥 8. Final Upgrade (this will differentiate you)

Add ONE of these:

Option A — "Ask My Resume" AI

- LLM that answers questions about your experience

Option B — Live Infra Demo

- Terraform plan preview
- Architecture visualization (visdoc)

🚀 9. Expected Outcome

With this setup:

✔️ Strong fit for **Staff / Principal roles**
✔️ Signals **platform-level thinking**
✔️ Shows **AI + DevOps convergence (rare)**

If you want next level

I can now:

✅ Build all 4 flagship repos (code + structure)
✅ Generate Terraform + CI/CD pipelines
✅ Create full portfolio website
✅ Add "Ask My Resume" AI system

Just say:
👉 *"build my full portfolio system end-to-end"*

Sources

- [ ]
👉 "build my full portfolio system end-to-end" and I wil add the portfolioprojects later

Good—this is the right move.
We'll build a **complete, production-grade portfolio system skeleton** so you can just plug projects in later.

No fluff—this is a **deployable architecture**.

🧠 SYSTEM OVERVIEW (what you're building)

```
GitHub Profile (README)
        ↓
Portfolio Website (Next.js)
        ↓
Project Registry (JSON / API)
        ↓
Visual + Demo Layer (VisDoc, diagrams)
        ↓
Optional: AI Assistant ("Ask my work")
```

📁 1. REPOSITORY STRUCTURE (create this first)

Create a **main portfolio repo**:

Bash
```
dattaram-portfolio/
```

Inside:

Bash

```
dattaram-portfolio/
│
├── frontend/          # Next.js site
├── backend/           # FastAPI (optional AI layer)
├── data/              # project registry
├── infra/             # Terraform (optional deploy)
├── .github/workflows/ # CI/CD
└── README.md
```

🌐 2. FRONTEND (Next.js Portfolio UI)

Use **Next.js**

Setup

Bash
```bash
npx create-next-app frontend
cd frontend
npm install
```

📄 Pages Structure

Bash
```bash
frontend/
├── pages/
│   ├── index.js        # Home
│   ├── projects.js     # Project listing
│   ├── architecture.js # Diagrams
│   └── contact.js
```

🧩 Basic Home Page

JavaScript
```javascript
// frontend/pages/index.js
export default function Home() {
  return (
    <div style={{ padding: "40px" }}>
      <h1>Dattaram Miruke</h1>
      <h3>Principal AI Platform Architect</h3>

      <p>
        AI Control Planes | DevOps | Kubernetes | AWS
      </p>

      <a href="/projects">View Projects →</a>
    </div>
  );
}
```

📊 3. PROJECT REGISTRY (key idea)

Instead of hardcoding projects → use a JSON file

Bash
```bash
data/projects.json
```

Example:

```JSON
[
  {
    "name": "AEGIS",
    "description": "AI Trust & Evaluation Framework",
    "tags": ["LLM", "RAG", "Governance"],
    "repo": "https://github.com/your-username/aegis",
    "demo": "",
    "architecture": ""
  }
]
```

Load it in UI

```JavaScript
// frontend/pages/projects.js
import projects from '../../data/projects.json';

export default function Projects() {
  return (
    <div style={{ padding: "40px" }}>
      <h1>Projects</h1>

      {projects.map((p, i) => (
        <div key={i}>
          <h2>{p.name}</h2>
          <p>{p.description}</p>
          <a href={p.repo}>Repo</a>
        </div>
      ))}
    </div>
  );
}
```

🧠 4. BACKEND (OPTIONAL but powerful)

Use **FastAPI**

This enables:

- “Ask my portfolio”
- Dynamic project data
- AI demos later

Minimal API

```Python
# backend/main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Portfolio API running"}
```

Run:

```bash
uvicorn main:app --reload
```

🎨 5. ARCHITECTURE PAGE (your differentiator)

```javascript
// frontend/pages/architecture.js
export default function Architecture() {
  return (
    <div style={{ padding: "40px" }}>
      <h1>System Architecture</h1>

      <pre>
{`User → API → LLM Control Plane → Agent Orchestrator
     → Tools → Memory → Observability → Policy`}
      </pre>

      <iframe
        src="http://37.27.97.75:8080/visdoc"
        width="100%"
        height="600px"
      />
    </div>
  );
}
```

⚙️ 6. CI/CD (GitHub Actions)

Create:

```bash
.github/workflows/deploy.yml
```

```yaml
name: Deploy Portfolio

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Install Node
        uses: actions/setup-node@v3
        with:
          node-version: 18

      - run: cd frontend && npm install && npm run build
```

🌍 7. DEPLOYMENT OPTIONS

Option A (FASTEST)

- Vercel (recommended)

Option B

- GitHub Pages

Option C (your style)

- AWS S3 + CloudFront + Terraform

🧱 8. TERRAFORM (optional infra)

```hcl
# infra/main.tf
provider "aws" {
  region = "us-west-2"
}

resource "aws_s3_bucket" "portfolio" {
  bucket = "dattaram-portfolio-site"
}
```

🧠 9. FUTURE (plug later)

You said you'll add projects later—good.

When ready, plug in:

🔴 DevOps Agent

🔴 AEGIS

🔴 RAG Platform

🔴 OpenBrain

Just update:

```Bash
data/projects.json
```

⚡ 10. LOCAL RUN

```Bash
cd frontend
npm run dev
```

Open:

```Bash
http://localhost:3000
```

🧠 WHAT YOU JUST BUILT

✔️ Modular portfolio system
✔️ Scalable (add projects anytime)

✔️ Supports AI + demos later
✔️ Matches Staff/Principal expectations

🔥 Next Step (high impact)

I recommend you do THIS next:

👉 Add **1 real project first (DevOps Agent)**
👉 Then wire:

- Terraform preview
- Architecture diagram

🚀 If you want next level

I can now:

1. Generate full **DevOps Agent repo (production-grade)**
2. Add **AI "Ask My Resume" assistant**
3. Add **real architecture visualization engine**
4. Deploy full system on AWS with Terraform

Just say:
👉 *"build first flagship project (DevOps Agent)"*

and we go deeper.

▢
at present I will set this up on hetzner server but will migrate it to aws shortly

That's a solid approach—Hetzner for speed/cost now, AWS later for production credibility. The key is to **design it so migration is trivial**, not a rewrite.

Let's set this up cleanly 👇

🧠 Deployment Strategy (important)

Build once, deploy anywhere:

```
App (Next.js + FastAPI)
        ↓
Docker (containerized)
        ↓
Runtime (Hetzner VM now → AWS later)
```

👉 If you **containerize correctly**, migration becomes:

- Hetzner VM → AWS EC2 / ECS / EKS
- No code changes

⚙️ 1. Hetzner Deployment (your current setup)

Assuming:

- Ubuntu server (you already run k3s, etc.)
- Public IP available

🔹 Install basics

```bash
sudo apt update
sudo apt install -y docker.io docker-compose git
```

🔹 Clone your repo

```bash
git clone https://github.com/your-username/dattaram-portfolio.git
cd dattaram-portfolio
```

🐳 2. Dockerize Everything (CRITICAL STEP)

Frontend Dockerfile

```dockerfile
# frontend/Dockerfile
FROM node:18

WORKDIR /app
COPY . .

RUN npm install
RUN npm run build

EXPOSE 3000

CMD ["npm", "start"]
```

Backend Dockerfile (optional)

```dockerfile
# backend/Dockerfile
FROM python:3.10

WORKDIR /app
COPY . .
```

```
RUN pip install fastapi uvicorn

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Docker Compose

```YAML
# docker-compose.yml
version: "3"

services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"

  backend:
    build: ./backend
    ports:
      - "8000:8000"
```

Run it

```Bash
docker-compose up -d
```

Open:

```Bash
http://your-server-ip:3000
```

🌐 3. Add Nginx (clean URLs + production feel)

Install:

```Bash
sudo apt install nginx
```

Config

```Nginx
server {
    listen 80;
    server_name your-domain.com;

    location / {
        proxy_pass http://localhost:3000;
    }

    location /api/ {
        proxy_pass http://localhost:8000/;
    }
}
```

Restart:

Bash
```bash
sudo systemctl restart nginx
```

🔐 4. Add HTTPS (important)

Use:

Bash
```bash
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx
```

🧠 5. Make Migration to AWS EASY (this is key)

Design decisions you must follow:

✅ Use:

- Docker (already done)
- Environment variables (no hardcoding)
- Stateless frontend
- External config (JSON/API)

🔴 When you move to AWS:

Option 1 (fast):

- EC2 + Docker (same setup)

Option 2 (better):

- ECS (Fargate)

Option 3 (best for you):

- EKS (since you already know k8s)

☁️ AWS Mapping

```
Hetzner VM → AWS EC2
Docker → ECS / EKS
Nginx → ALB / Ingress
Static assets → S3 + CloudFront
```

⚡ 6. Bonus (since you run k3s)

You can deploy this directly:

Bash

```
kubectl apply -f deployment.yaml
```

Later:

- Move same manifests → EKS

🧠 7. Reality Check (important)

Don't overbuild infra now.

Focus:

✔️ Get portfolio live
✔️ Add 1–2 strong projects
✔️ Show working demos

🔥 Recommended Path

Phase 1 (NOW — Hetzner)

- Docker + Nginx
- Domain + HTTPS
- Portfolio live

Phase 2 (1–2 weeks)

- Add real projects
- Add architecture diagrams
- Add visdoc integration

Phase 3 (AWS Migration)

- Terraform infra
- Deploy to EKS or ECS
- Add CI/CD

🚀 If you want next step

I can generate:

✅ Kubernetes manifests (ready for k3s + EKS)
✅ Terraform for AWS migration
✅ Full CI/CD pipeline (GitHub Actions → deploy)
✅ Observability stack (Prometheus + Grafana for portfolio)

Just say:
👉 *"prepare k8s + terraform for migration"*

I just wasnt to shw the github repos for now ; do notmkae ti moe complicated

Good—then don't overengineer this. You don't need servers, Docker, or Hetzner for what you want right now.

👉 If your goal is **just to show GitHub repos**, the simplest and best setup is:

✅ What you should do (minimal, effective)

1. Use your GitHub Profile as the "website"

On GitHub:

Create this repo:

```Bash
DattaramMiruke/DattaramMiruke
```

Add:

```Bash
README.md
```

👉 This becomes your **main portfolio page**

---

2. Add your projects as links (even if empty for now)

Use this clean version:

```Markdown
# 👋 Dattaram Miruke

### Principal AI Platform Architect | DevOps | Kubernetes | AWS

---

## 🚀 Portfolio

### 🧠 AEGIS — AI Trust Framework
👉 https://github.com/DattaramMiruke/aegis

---

### ⚙ DevOps Agent System
👉 https://github.com/DattaramMiruke/devops-agent
```

```
---

### 📊 RAG Platform
👉 https://github.com/DattaramMiruke/rag-platform

---

### 🧠 OpenBrain Memory System
👉 https://github.com/DattaramMiruke/openbrain

---

## ⚙ Stack
AWS | Kubernetes | Terraform | Python | Go

---

## 📫 Contact
- Email: dattamiruke@gmail.com
```

3. Pin repositories (VERY IMPORTANT)

Go to your profile → “Customize pins”

Pin:

- aegis
- devops-agent
- rag-platform
- openbrain

👉 This creates a **visual portfolio grid**

🧠 That’s it. Seriously.

No:
❌ Hetzner
❌ Docker
❌ Next.js
❌ Backend

🔥 Why this is actually better (right now)

Recruiters:

- Don’t visit your server
- Don’t care about deployment infra (initially)
- Do care about:
  - Clean profile
  - Clear projects

- Good naming

⚡ When to upgrade later

Only after you have:
✔️ 2–3 real projects
✔️ Some code / demos

Then build:

- Website
- AI assistant
- Infra demos

🧠 Brutal truth

Right now your bottleneck is:
👉 **projects, not platform**

🚀 Next step (focused)

Do this next:

1. Create empty repos:

```Bash
aegis
devops-agent
rag-platform
openbrain
```

2. Add simple README in each:

```Markdown
# Project Name

Work in progress.
```

3. Link them in your profile

If you want, I'll next:
👉 Write **perfect README for each repo (high-impact, recruiter-ready)**

Just say:
"create repo READMEs"

▢
"create repo READMEs"

Perfect—this is exactly the right next step.

We'll create **high-signal, recruiter-friendly README.md files** for each repo.
Each one will:

- Clearly state **problem** → **solution** → **architecture**
- Show **platform-level thinking**
- Be concise but **technically credible**

You can copy-paste these directly into each repo.

🧠 1. `aegis` — AI Trust & Evaluation Framework

```Markdown
# 🧠 AEGIS — AI Trust, Evaluation & Governance Framework

## 🚀 Overview
AEGIS is a framework for **evaluating, governing, and monitoring agentic AI systems**.

It provides a **trust layer** for LLM-based systems, enabling:
- Evaluation (quality, correctness, hallucination detection)
- Governance (policy enforcement, RBAC)
- Observability (metrics, scoring, telemetry)

---

## ❗ Problem

Modern AI systems (LLMs, RAG, agents) lack:
- Standardized evaluation pipelines
- Policy enforcement mechanisms
- Observability into model behavior

This leads to:
- Hallucinations
- Unsafe outputs
- Lack of auditability

---

## ✅ Solution

AEGIS introduces a **structured evaluation + governance layer**:

- 📊 Evaluation Engine (RAGAS, DeepEval-style scoring)
- 🔐 Policy Engine (OPA-based guardrails)
- 📈 Observability Layer (metrics, traces)
- 🧪 Test Harness for agent workflows

---

## 🏗️ Architecture
```

User → AI System → AEGIS Layer
├── Evaluation Engine
├── Policy Engine (OPA)

```
├── Observability (metrics/logs)
└── Scoring + Reporting
```

```
---

## ⚙ Tech Stack

- Python (FastAPI)
- LLM frameworks (LangChain / LangGraph)
- Evaluation: RAGAS / custom metrics
- Policy: OPA (Open Policy Agent)
- Observability: OpenTelemetry, Prometheus

---

## 📌 Status

🚧 Work in progress — building core evaluation pipelines and policy engine

---

## 🎯 Goal

Provide a **production-grade trust layer for agentic AI systems**

---

## 📎 Future

- Multi-agent evaluation workflows
- Benchmark datasets
- CI/CD integration for AI evaluation
```

⚙️ 2. `devops-agent` — Infra Generation System

```Markdown
# ⚙ DevOps Agent — AI-Powered Infrastructure Generation

## 🚀 Overview

DevOps Agent is an AI system that converts **natural language intent → infrastructure + CI/CD
pipelines**.

---

## ❗ Problem

Designing infrastructure is:
- Manual
- Error-prone
- Inconsistent across teams

---

## ✅ Solution

An AI agent that:
- Interprets user intent
- Generates Terraform code
- Creates CI/CD pipelines
- Validates architecture

---

## 🧮 Architecture
```

User Input → LLM Planner → Infra Generator
→ Terraform → CI/CD (GitHub Actions)

→ Validation Engine

```
---

## ⚙️ Features

- 🧠 Intent parsing → structured infra plan
- ⚙️ Terraform generation
- 🔁 CI/CD pipeline creation
- 📊 Architecture scoring (planned)

---

## ⚙️ Tech Stack

- Python (FastAPI)
- LLM APIs (OpenAI / Bedrock)
- Terraform
- GitHub Actions
- JSON/YAML spec generation

---

## 📌 Status

🚧 Early-stage — building intent parsing and Terraform generation

---

## 🎯 Goal

Enable **fully automated DevOps workflows using AI agents**

---

## 📎 Future

- Multi-cloud support (AWS, GCP, Azure)
- Kubernetes manifest generation
- Cost optimization engine
```

📊 3. `rag-platform` — Document Intelligence System

```Markdown
# 📊 RAG Platform — Document Intelligence & Retrieval System

## 🚀 Overview

A scalable platform for **document ingestion, embedding, retrieval, and evaluation** using RAG
(Retrieval-Augmented Generation).

---

## ❗ Problem

LLMs alone:
- Lack domain knowledge
- Cannot access real-time data
- Produce hallucinated responses

---

## ✅ Solution

RAG platform that:
- Ingests documents
- Creates embeddings
- Performs semantic retrieval
- Augments LLM responses with context

---
```

```
## 🧱 Architecture
```

Documents → Ingestion → Embeddings → Vector Store
↓
User Query → Retriever → Context → LLM → Response

```
---

## ⚙️ Features

- 📥 Multi-source ingestion (PDF, APIs, logs)
- 🔎 Semantic search (vector + metadata)
- 🧠 PromptBuilder (structured prompts)
- 📊 Evaluation pipelines (planned)

---

## ⚙️ Tech Stack

- Python
- Vector DB (FAISS / OpenSearch / pgvector)
- LLM frameworks (LangChain)
- FastAPI

---

## 📌 Status

🚧 Core ingestion + retrieval pipeline under development

---

## 🎯 Goal

Build a **production-ready RAG system for enterprise AI use cases**

---

## 📎 Future

- Streaming ingestion
- Multi-tenant support
- Evaluation + observability integration
```

🧬 4. `openbrain` — AI Memory System

Markdown

```
# 🧬 OpenBrain — Persistent Memory System for AI Agents

## 🚀 Overview

OpenBrain is a **persistent memory system for LLMs and agents**, enabling long-term context and
knowledge retention.

---

## ❗ Problem

LLMs are stateless:
- No long-term memory
- Limited context window
- No knowledge accumulation

---

## ✅ Solution

A hybrid memory system:
- Vector embeddings (semantic recall)
```

```
- Structured metadata (context tracking)
- Multi-agent memory sharing

---

## 🧮 Architecture
```

Input → Embedding → Vector Store
→ Metadata Store

Agent → Retrieve → Context → Reason → Store back

```
---

## ⚙️ Features

- 🧠 Long-term memory storage
- 🔍 Context-aware retrieval
- 🔁 Continuous knowledge enrichment
- 🤝 Multi-agent shared memory

---

## ⚙️ Tech Stack

- Python
- PostgreSQL + pgvector
- Embedding models
- FastAPI

---

## 📌 Status

🚧 Designing memory schema + retrieval pipelines

---

## 🎯 Goal

Enable **stateful, context-aware AI systems**

---

## 📎 Future

- Graph-based memory
- Memory ranking/scoring
- Cross-session learning
```

🔥 What you should do NOW

1. Create these repos on GitHub
2. Add each README
3. Pin them on your profile

🧠 Why this works

These READMEs:

- Show **system design thinking**
- Avoid toy-project feel

- Align with **Staff/Principal roles**
- Make you look like a **builder, not a learner**

🚀 Next step (high ROI)

After this, I recommend:

👉 Add **one simple working feature** in ONE repo
(e.g., DevOps Agent → basic Terraform generator)

If you want, next I can:
👉 Turn ONE repo into a **fully working demo (end-to-end code)**

- [ ]
so what do we have now

Right now, you've built the **foundation of a hiring-ready GitHub portfolio**—even if no real code is in place yet.

Let's be precise about what you *actually have* 👇

🧠 What you have (current state)

1. Profile-level portfolio (your homepage) on GitHub

- A structured `README.md`
- Clear positioning:
  👉 *Principal AI Platform Architect*
- Clean sections:
  - What you build
  - Projects
  - Stack
  - Contact

👉 This acts as your **landing page**

---

2. 4 flagship project repos (portfolio skeleton)

You now have:

🔴 AEGIS

- AI governance + evaluation system

⚙️ DevOps Agent

- Infra generation via AI

📊 RAG Platform

- Document intelligence + retrieval

🧬 OpenBrain

- Persistent AI memory system

👉 Each has:

- Problem statement
- Architecture
- Tech stack
- Direction

---

3. High-quality READMEs (this is the real value)

These are not:
❌ Tutorials
❌ Random experiments

They are:
✅ System-level designs
✅ Architect-level thinking
✅ Production intent

---

4. A coherent story (this is the biggest win)

Before:
❌ Disconnected skills

Now:

```
AI Platform Systems
   ├── AEGIS (trust layer)
   ├── DevOps Agent (infra automation)
   ├── RAG Platform (data + retrieval)
   └── OpenBrain (memory layer)
```

👉 This looks like a **platform ecosystem**, not random repos

🔥 What you do NOT have yet (important)

Let's be honest:

Missing:

- ❌ Working code
- ❌ Live demos
- ❌ Real outputs
- ❌ Metrics / results

👉 That's fine—for now

🧠 What this stage is called

You are in:

👉 **"Signal Creation Phase"**

Goal:

> Make recruiters think: *"This person designs real systems"*

⚡ What a recruiter sees (today)

If they open your profile:

They think:

- "Clear direction"
- "Senior-level thinking"
- "Interesting systems"

But also:

- "Need proof (code/demo)"

🚀 Next step (highest ROI)

You only need **ONE thing now**:

👉 Turn **ONE repo into a working demo**

Recommended order:

Step 1:

Pick:
👉 **DevOps Agent**

Step 2:

Add ONE feature:

Example:

- Input: “create AWS web app”
- Output: basic Terraform file

Step 3:

Update README:

Add:

```Markdown
## Demo

Input:
"Create S3 + CloudFront static site"

Output:
(terraform snippet)
```

🧠 Strategy (very important)

Don’t build everything.

Do:

```
1 repo → real
3 repos → conceptual
```

🔥 Your current position

You’ve moved from:

❌ “Engineer with experience”

➡️ to:

✅ “Architect building an AI platform ecosystem”

🚀 If you want next move

I can take **DevOps Agent** and:

- Generate working code (FastAPI + LLM)
- Add Terraform generation logic
- Add CLI + API
- Make it demo-ready in 1–2 hours

Just say:
👉 *"build DevOps Agent MVP"*

That's the moment your profile becomes **seriously competitive**.

> ☐
> No I mean the github page and the related contents

Got it—you're asking specifically:

👉 **"What does my GitHub page look like now (structure + contents)?"**

Here's exactly what you have (or should have) at this point 👇

🧠 Your GitHub Profile = Your Portfolio Homepage

On GitHub, your profile page is now structured like this:

🧱 1. TOP SECTION — Profile README (Main Content)

This is the **big block at the top of your profile**

It contains:

```
👋 Dattaram Miruke
Principal AI Platform Architect | DevOps | Kubernetes | AWS

[Short intro]

🚀 Portfolio
- AEGIS → link
- DevOps Agent → link
- RAG Platform → link
- OpenBrain → link

⚙️ Stack
📫 Contact
```

👉 This is your **homepage content**
👉 This is what recruiters read first

🧩 2. PINNED REPOSITORIES (Visual Grid)

Right below the README, GitHub shows:

```
[ AEGIS ]   [ DevOps Agent ]   [ RAG Platform ]
[ OpenBrain ]   [ (optional 2 more) ]
```

Each box shows:

- Repo name
- Description (from README first line)
- Language (later when you add code)

👉 This acts like your **project showcase cards**

🗂️ 3. EACH REPO (when clicked)

Each project opens to its own page:

Example:

```
github.com/DattaramMiruke/devops-agent
```

And shows:

```
README.md

Title
Problem
Solution
Architecture
Tech Stack
Future
```

👉 This is your **deep-dive page per project**

🧠 4. OVERALL FLOW (how someone navigates)

```
Profile Page
   ↓
Reads README (top summary)
   ↓
Sees pinned repos
   ↓
Clicks a project
   ↓
Reads detailed README
```

🔥 What you have RIGHT NOW (state)

✅ You HAVE:

✔️ Profile README (portfolio overview)
✔️ 4 strong project concepts
✔️ Structured repo READMEs
✔️ Clear technical positioning

❌ You DO NOT HAVE (yet):

❌ Code
❌ Demos
❌ Live outputs

👉 That's okay at this stage

👀 What it looks like to a recruiter

First impression (top section):

> "Senior/Principal-level AI + DevOps architect"

After scrolling:

> "Has a structured platform vision"

After clicking a repo:

> "Thinks in systems, not scripts"

⚡ What makes your profile already strong

Compared to most profiles:

- Not random repos
- Not tutorial clutter
- Not generic

Instead:
👉 **Coherent system narrative**

🧠 Small but IMPORTANT improvements (do these now)

1. Repo descriptions (very important)

Edit each repo (top right → settings)

Add 1 line like:

- AEGIS →
  `AI evaluation, governance, and trust framework for LLM systems`
- DevOps Agent →
  `AI system for generating infrastructure and CI/CD pipelines`

2. Add topics (tags)

In each repo:

```
ai
llm
devops
terraform
kubernetes
rag
```

👉 Improves discoverability

---

3. Pin repos (if not done)

Max 6 → keep it clean

🧠 Final mental model

Your GitHub is now:

```
Landing Page (README)
   ↓
Project Cards (Pinned Repos)
   ↓
System Designs (Repo READMEs)
```

🚀 Where you are in the journey

You are here:

```
[✔] Portfolio structure
[✔] Project narrative
[ ] Working demo (next)
[ ] Production-level proof
```

🔥 One-line summary

👉 You now have a **clean, architect-level GitHub portfolio skeleton** that is ready to convert into a high-impact profile once you add even a small amount of working code.

If you want, next I can:
👉 Show you exactly how your profile would look visually (mock layout)
👉 Or upgrade one repo into a working demo (fastest impact)

Just tell me 👍

▢
shiow me the repo streuctures and each file in it \

Perfect—this is what you actually need now: **clean repo skeletons you can create immediately**. No overengineering, just **correct structure + meaningful files**.

Below are the **exact folder structures + file contents** for each repo.

🧠 1. `aegis` — Repo Structure

Bash
```
aegis/
├── README.md
├── docs/
│   └── architecture.md
├── src/
│   ├── main.py
│   ├── evaluator.py
│   └── policy.py
├── tests/
│   └── test_basic.py
└── requirements.txt
```

📄 `README.md`

👉 (Use the one I already gave you)

📄 `docs/architecture.md`

Markdown
```
# Architecture

AEGIS acts as a middleware layer between users and AI systems.

## Flow

User → LLM → AEGIS Layer → Output

## Components

- Evaluation Engine
- Policy Engine (OPA)
- Observability Layer
- Scoring Engine
```

📄 `src/main.py`

Python
```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "AEGIS running"}
```

📄 src/evaluator.py

Python
```python
def evaluate_response(response: str):
    score = len(response)  # placeholder logic
    return {"score": score}
```

📄 src/policy.py

Python
```python
def check_policy(response: str):
    if "bad" in response.lower():
        return False
    return True
```

📄 requirements.txt

```
fastapi
uvicorn
```

⚙️ 2. devops-agent — Repo Structure

Bash
```bash
devops-agent/
├── README.md
├── src/
│   ├── main.py
│   ├── planner.py
│   └── terraform_generator.py
├── examples/
│   └── sample_output.tf
└── requirements.txt
```

📄 src/main.py

Python
```python
from fastapi import FastAPI
from planner import plan

app = FastAPI()

@app.get("/")
def root():
    return {"message": "DevOps Agent running"}

@app.get("/generate")
def generate(input: str):
    return plan(input)
```

📄 src/planner.py

Python
```python
from terraform_generator import generate_tf

def plan(user_input: str):
```

```python
    return {
        "input": user_input,
        "terraform": generate_tf(user_input)
    }
```

📄 src/terraform_generator.py

```python
def generate_tf(user_input: str):
    return """
resource "aws_s3_bucket" "example" {
  bucket = "my-bucket"
}
"""
```

📄 examples/sample_output.tf

```hcl
resource "aws_s3_bucket" "example" {
  bucket = "my-bucket"
}
```

📊 3. rag-platform — Repo Structure

```bash
rag-platform/
├── README.md
├── src/
│   ├── ingest.py
│   ├── retriever.py
│   └── main.py
├── data/
│   └── sample.txt
└── requirements.txt
```

📄 src/ingest.py

```python
def ingest_document(text: str):
    return text.split()
```

📄 src/retriever.py

```python
def retrieve(query: str, docs):
    return [d for d in docs if query in d]
```

📄 src/main.py

```python
from fastapi import FastAPI

app = FastAPI()
```

```python
@app.get("/")
def root():
    return {"message": "RAG Platform running"}
```

📄 data/sample.txt

```
This is a sample document for testing retrieval.
```

🧬 4. openbrain — Repo Structure

```bash
openbrain/
├── README.md
├── src/
│   ├── memory.py
│   ├── store.py
│   └── main.py
└── requirements.txt
```

📄 src/memory.py

```python
memory_store = []

def add_memory(item):
    memory_store.append(item)

def get_memory():
    return memory_store
```

📄 src/store.py

```python
def store_embedding(data):
    return {"embedding": hash(data)}
```

📄 src/main.py

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "OpenBrain running"}
```

🧠 What this gives you RIGHT NOW

Each repo now has:

✔️ Real folder structure
✔️ Real code (minimal but valid)
✔️ Logical separation of components
✔️ Expandable architecture

🔥 Why this is important

This avoids:

❌ Empty repos
❌ "Just README" profiles

Instead you now have:

👉 **Executable skeleton systems**

⚡ What to do next (10 min task)

For each repo:

```bash
git init
git add .
git commit -m "Initial structure"
git push
```

🚀 After this

Your GitHub will show:

- Real repos
- Real code
- Real structure

👉 Even without full features, this already signals:
"This person builds systems"

If you want next step

I can now:

👉 Turn ONE repo (DevOps Agent) into a **real working AI system with LLM integration**

Just say:
"upgrade devops-agent to real MVP"

NBo the profile page

Got it—you want the **actual structure/content of the *profile page repo*** (not the project repos).

This is the special repo that powers your GitHub homepage.

🧠 Profile Repo Structure

On GitHub you need:

```bash
DattaramMiruke/
└── README.md
```

👉 That's it.
No folders needed (you *can* add assets later, but keep it simple now).

📄 `README.md` (FULL CONTENT — copy this)

This is your **entire profile page content**:

```markdown
# 👋 Dattaram Miruke

### Principal AI Platform Architect | DevOps | Kubernetes | AWS

📍 Sacramento, CA
📧 dattamiruke@gmail.com

---

## 🚀 What I Build

I design and build **AI + cloud platform systems**:

- 🧠 AI Control Planes (multi-LLM, routing, policy)
- ⚙️ DevOps Automation (Terraform, CI/CD)
- 📊 RAG Systems (retrieval + evaluation)
- 🧬 AI Memory Systems (persistent context)

---

## 🔴 Portfolio

### 🧠 AEGIS — AI Trust & Evaluation Framework
https://github.com/DattaramMiruke/aegis

---

### ⚙️ DevOps Agent — Infra Generation System
https://github.com/DattaramMiruke/devops-agent

---

### 📊 RAG Platform — Document Intelligence
https://github.com/DattaramMiruke/rag-platform

---

### 🧬 OpenBrain — AI Memory System
https://github.com/DattaramMiruke/openbrain
```

```
---

## 🧮 Architecture Focus
```

User → API → LLM Control Plane → Agent Orchestrator
→ Tools → Memory → Observability → Policy

```
---

## ⚙ Stack

AWS | Kubernetes | Terraform | Python | Go | LLM | RAG

---

## 📊 GitHub Stats

![Stats](https://github-readme-stats.vercel.app/api?username=DattaramMiruke&show_icons=true)

---

## 📫 Contact

- Email: dattamiruke@gmail.com
- GitHub: https://github.com/DattaramMiruke
```

🧠 How this appears on your profile

When someone visits:

```
https://github.com/DattaramMiruke
```

They see:

```
[Your README content here]

↓ below it

Pinned Repositories (grid)
- aegis
- devops-agent
- rag-platform
- openbrain
```

⚡ Optional (only if you want slightly better look)

You can add a folder:

```Bash
DattaramMiruke/
├── README.md
└── assets/
    └── banner.png
```

Then reference:

```Markdown
![Banner](./assets/banner.png)
```

🔥 What matters most (keep it simple)

For now:

✔️ One clean README
✔️ Clear project links
✔️ Pinned repos

🚀 You're done when:

- Repo `DattaramMiruke` exists
- README.md added
- Repos pinned

🧠 Final clarity

👉 Your "profile page" is **just one README.md file rendered by GitHub**

No:

- HTML
- Backend
- Hosting

If you want next:
👉 I can make this README more visually rich (badges, layout, sections) without making it messy
👉 Or show a side-by-side "before vs recruiter-optimized" version

hkaanturgut_openclaw-job-search-agent.pdf
PDF

Hcheck this screen dump ; How do I take a web site page dump

Got it—you're asking how to create a **clean "web page dump" like the GitHub README view** you showed in your screenshot hkaanturgut_openclaw-job-search… .
That screenshot is essentially:
👉 **Rendered Markdown (README) + page capture**

🧠 What you're trying to do

You want something like:

- Full GitHub repo page (README + visuals)

- Looks like a **website page**
- Can be shared / saved

✅ 3 BEST ways to take a “web page dump”

🥇 Option 1 — Save as PDF (BEST & easiest)

This is exactly what your screenshot looks like.

Steps (Chrome / Firefox / Edge):

1. Open page (example):

```
https://github.com/hkaanturgut/openclaw-job-search-agent
```

2. Press:

```Bash
Ctrl + P
```

3. Select:

```
Destination → Save as PDF
```

4. Enable:

- ✅ Background graphics
- ✅ More settings → Scale 100%

5. Save

👉 Output = **clean full page like your file**
hkaanturgut_openclaw-job-search…

🥈 Option 2 — Full Page Screenshot (visual dump)

Use browser tools:

Chrome DevTools:

1. Press:

```Bash
Ctrl + Shift + I
```

2. Then:

```Bash
```

```
Ctrl + Shift + P
```

3. Type:

```
Capture full size screenshot
```

👉 Output:

- One long PNG image of entire page

🥉 Option 3 — Download as HTML (true "web dump")

This saves the actual page:

Right-click:

```
Save As → Webpage, Complete
```

You get:

```bash
page.html
page_files/
```

👉 This is closest to:
✔️ "offline website copy"

⚡ Bonus (best for GitHub repos specifically)

Clone + render locally

```bash
git clone https://github.com/hkaanturgut/openclaw-job-search-agent
```

Then:

- Open `README.md` in VS Code
- Preview it

👉 This gives **clean markdown rendering without GitHub UI clutter**

🧠 What your screenshot shows (important insight)

From your dump hkaanturgut_openclaw-job-search… :

- GitHub repo page
- README rendered like a website
- Sections:
    - Demo video

- What this does
- Architecture diagram
- Telegram output
- Setup steps

👉 This is just:
Markdown → rendered by GitHub → exported as PDF

🔥 Recommended for YOU

Since you're building a portfolio:

Use:

👉 PDF export (Option 1)

Why:

- Clean
- Shareable
- Looks professional
- Matches recruiter expectations

🚀 Pro Tip (next level)

For your own repos:

1. Build strong README
2. Then export:

```bash
Ctrl + P → Save as PDF
```

👉 Now you have:

- Portfolio website (GitHub)
- - Downloadable “project doc”

🧠 Simple rule

Goal	Method
Share nicely	PDF
Show visually	Screenshot
Keep offline site	HTML save

If you want

I can:
👉 Convert your GitHub README into a **perfect "PDF-style presentation layout"** like the one you showed
👉 Add sections (demo, architecture, visuals, flow)

Just say:
"convert my README to showcase format"

Sources

hkaanturgut (Kaan Turgut).htm
File

Do you need js css files?>

Short answer: **No — you usually do NOT need JS/CSS files** (depends on what you want).

Let's break it down cleanly 👇

🧠 What are you trying to achieve?

1. 📄 "Clean page dump" (like your screenshot)

👉 **NO JS/CSS needed**

- Use:
  - **Save as PDF**
  - or **Full page screenshot**

✔️ Includes:

- Layout (rendered already)
- Fonts/colors baked in
- Images

❌ Does NOT need:

- `.js`
- `.css`

👉 This is what you already have in your dump

---

2. 🌐 "Offline working website copy"

👉 **YES — you need JS/CSS**

If you want:

- Buttons to work
- Animations
- Interactive UI
- Navigation

Then you must save:

```
index.html
+ CSS files
+ JS files
+ images/assets
```

3. 📄 GitHub README case (your example)

GitHub pages are:

- Markdown → rendered server-side

👉 So when you:

- Save as PDF ✅ works perfectly
- Screenshot ✅ works perfectly

👉 You do NOT need:

- JS
- CSS

Because:
✔️ GitHub already rendered everything

⚡ Simple decision table

Goal	Need JS/CSS?
PDF / share doc	❌ No
Screenshot	❌ No

Goal	Need JS/CSS?
Offline interactive site	✅ Yes
GitHub README dump	❌ No

🔥 Practical recommendation (for YOU)

Since you're:

- Building portfolio
- Sharing projects

👉 Use:

✔️ PDF export (best)

✔️ Screenshot (optional)

🧠 When WOULD you need JS/CSS?

Only if you're doing:

- Cloning a site
- Hosting locally
- Editing UI
- Building a demo

🚀 Pro tip (important)

If your goal is:

👉 “Portfolio-grade project showcase”

Then don’t dump raw site.

Instead:

- Export README → PDF
- OR build your own clean page (React / static)

If you want next step

I can help you:
👉 Turn your GitHub repo into:

- **Clean portfolio website (React or static)**
- **Downloadable PDF showcase**
- **Architecture + flow diagrams included**

Just say:
“build portfolio version of my repo”

ok use the pdf file and create website page

Good — this is exactly the right move (PDF → clean portfolio page).
Since you didn’t upload the PDF yet, I’ll give you a **ready-to-use production template** + workflow so you can plug your PDF content into a proper website immediately.

🧠 What we’re building

A **clean portfolio project page** from your PDF:

- Structured (not dump)
- Recruiter-friendly
- GitHub-ready
- Extendable later (React / full portfolio)

⚙️ Step 1 — Extract content from PDF

Use one of these:

Option A (fastest)

- Open PDF → Copy sections manually

Option B (better)

```bash
pdftotext yourfile.pdf output.txt
```

What to extract:

- Title
- Overview
- Architecture

- Features
- Code snippets
- Diagrams (screenshots)

🧱 Step 2 — Website structure

We'll use **simple static HTML (best first version)**

```
project/
 ├── index.html
 ├── style.css
 └── assets/
     ├── images/
     └── diagrams/
```

💻 Step 3 — Production-ready HTML template

Use this directly 👇

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Project Portfolio</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
  <h1>🚀 Project Title</h1>
  <p>Short one-line summary of your project</p>
</header>

<section>
  <h2>📌 Overview</h2>
  <p>
    Paste your PDF summary here. Keep it clean and structured.
  </p>
</section>

<section>
  <h2>🧱 Architecture</h2>
  <pre>
+-------------+
|   Frontend  |
+-------------+
       |
+-------------+
|   Backend   |
+-------------+
       |
+-------------+
|  Database   |
+-------------+
  </pre>
</section>

<section>
  <h2>⚙ Tech Stack</h2>
  <ul>
    <li>AWS</li>
    <li>Kubernetes</li>
    <li>Terraform</li>
    <li>Python / Go</li>
```

```
  </ul>
</section>

<section>
  <h2>✨ Features</h2>
  <ul>
    <li>Feature 1</li>
    <li>Feature 2</li>
    <li>Feature 3</li>
  </ul>
</section>

<section>
  <h2>📊 Diagrams</h2>
  <img src="assets/images/diagram.png" alt="Architecture Diagram">
</section>

<section>
  <h2>💻 Code Snippet</h2>
  <pre><code>
# Example Terraform snippet
resource "aws_s3_bucket" "example" {
  bucket = "my-bucket"
}
  </code></pre>
</section>

<section>
  <h2>📈 Learnings / Impact</h2>
  <p>
    What you achieved, performance gains, scale, etc.
  </p>
</section>

<footer>
  <p>© Your Name</p>
</footer>

</body>
</html>
```

🎨 Step 4 — CSS (clean, recruiter-grade)

CSS

```
body {
  font-family: Arial, sans-serif;
  margin: 40px;
  background: #f5f5f5;
  color: #333;
}

header {
  text-align: center;
  margin-bottom: 40px;
}

section {
  background: white;
  padding: 20px;
  margin-bottom: 20px;
  border-radius: 10px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

h2 {
  color: #2c3e50;
}

pre {
  background: #222;
  color: #0f0;
  padding: 15px;
```

```
  border-radius: 5px;
  overflow-x: auto;
}

img {
  max-width: 100%;
}
```

🚀 Step 5 — Upgrade (optional but recommended)

Once basic works:

Add:

- ✅ Navigation sidebar
- ✅ Dark mode toggle
- ✅ Copy code button
- ✅ Tabs (Architecture / Code / Docs)

🧠 How YOU should structure content (important)

Don't dump PDF blindly.

Transform it like this:

❌ Bad

Raw paragraphs

✅ Good

- Problem
- Solution
- Architecture
- Stack
- Tradeoffs
- Results

🔥 Next step (I can do this for you)

Upload your PDF and I will:

✔️ Extract content
✔️ Structure it professionally
✔️ Generate full website (HTML + CSS + optional React version)
✔️ Add:

- Architecture diagrams (clean ASCII or visual)

- DevOps flows
- System design sections

If you want higher level:

👉 I can convert it into:

- Full **portfolio site**
- Integrated with your **DevOps Agent / AEGIS projects**
- Hosted on **GitHub Pages / S3 + CloudFront**

Just upload the PDF 👍

▢
show me the repo structure

Perfect—since you're turning a **PDF → website page**, here's the **exact repo structure you should use** (simple, clean, no overengineering).

🧱 📁 Repo Structure (Final)

Bash
```
portfolio-project/
├── index.html          # Main web page (your PDF converted)
├── style.css           # Styling
├── README.md           # GitHub repo description
└── assets/
    ├── images/         # Screenshots from PDF
    │   ├── architecture.png
    │   ├── ui.png
    │   └── flow.png
    └── pdf/
        └── original.pdf
```

📄 1. `index.html`

👉 Your **actual website page**

Contains:

- Title
- Overview
- Architecture
- Features
- Code snippets

- Diagrams (from `assets/images`)

🎨 2. `style.css`

👉 Makes it look like a **real portfolio page (not raw HTML)**

📘 3. `README.md`

👉 This is what shows on GitHub

Keep it short:

```Markdown
# Project Name

Portfolio version of the project.

## 🌐 View Page
Open `index.html` locally or deploy via GitHub Pages.

## 📄 Original PDF
See `/assets/pdf/original.pdf`
```

🖼️ 4. `assets/images/`

👉 Extract images from your PDF:

Examples:

- Architecture diagrams
- Screenshots
- Flow charts

📄 5. `assets/pdf/original.pdf`

👉 Keep original for:

- Reference
- Sharing
- Credibility

🚀 How to run locally

```Bash
cd portfolio-project
open index.html
```

OR:

```Bash
```

```
python3 -m http.server 8080
```

Open:

```Bash
http://localhost:8080
```

🌐 Optional (GitHub Pages — 1 click deploy)

In GitHub:

1. Go to repo → Settings
2. Pages
3. Select:

```
Deploy from branch → main → /root
```

Then your site is live:

```
https://your-username.github.io/portfolio-project/
```

🧠 Why this structure works

✔️ Simple (no React, no backend)
✔️ Fast to build
✔️ Easy to deploy
✔️ Looks professional
✔️ Can evolve later

🔥 What NOT to do

❌ Don't add:

- backend/
- docker/
- terraform/

👉 Not needed for this stage

🚀 If you want next step

I can:
👉 Convert your **actual PDF into fully filled `index.html`**
👉 With clean sections + diagrams + formatting

Just upload the PDF 👍

▢
I do not wanty the website in a bucket, it willbehosted through hetzner server for now

Perfect—that actually simplifies everything.

👉 You don't need S3, GitHub Pages, or anything fancy.
You just need a **static site served from your Hetzner server**.

🧠 Final Repo Structure (for YOUR setup)

Keep it clean and minimal:

Bash
```
portfolio-project/
├── index.html
├── style.css
├── README.md
└── assets/
    ├── images/
    └── pdf/
        └── original.pdf
```

👉 This works **directly on Hetzner + browser**

⚙️ How to host on Hetzner (2 simple options)

✅ Option 1 — EASIEST (no setup, quick test)

Bash
```
cd portfolio-project
python3 -m http.server 8080
```

Open:

```
http://your-server-ip:8080
```

👉 Done.
(No nginx, no config)

✅ Option 2 — Proper setup (recommended)

Use **Nginx** (clean URLs + production feel)

---

1. Install nginx

Bash
```
sudo apt update
sudo apt install nginx
```

2. Copy your site

```bash
sudo mkdir -p /var/www/portfolio
sudo cp -r portfolio-project/* /var/www/portfolio/
```

3. Configure nginx

Edit:

```bash
sudo nano /etc/nginx/sites-available/portfolio
```

Paste:

```nginx
server {
    listen 80;
    server_name _;

    root /var/www/portfolio;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

4. Enable site

```bash
sudo ln -s /etc/nginx/sites-available/portfolio /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

5. Open in browser

```
http://your-server-ip
```

👉 Your portfolio page is live

🔐 Optional (but recommended)

Add HTTPS:

```bash
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx
```

🧠 How this fits your goal

You now have:

✔️ GitHub profile → links to repos
✔️ Repo → contains README + website files
✔️ Hetzner → hosts your project page

🔥 Clean workflow (important)

```
GitHub Repo
   ↓
Contains index.html + assets
   ↓
Pull to Hetzner
   ↓
Served via nginx
```

⚡ When you update content

Just:

```bash
git pull
sudo cp -r * /var/www/portfolio/
sudo systemctl reload nginx
```

🚀 Reality check

This is already **production-grade enough** for:

- Portfolio
- Recruiter demos
- Project showcasing

If you want next step

I can now:

👉 Convert your **actual PDF → finished index.html (fully filled)**
👉 Add:

- clean layout
- architecture diagrams
- sections like your screenshot

Just upload the PDF 👍

---

gibe me each filecntnets as doqwnloable canvas

```
// ==========================
// FILE: index.html
// ==========================
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Portfolio Project</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
  <h1>🚀 Project Title</h1>
  <p>Short one-line summary of your project</p>
</header>

<section>
  <h2>📌 Overview</h2>
  <p>
    Replace this with content extracted from your PDF.
  </p>
</section>

<section>
  <h2>🧮 Architecture</h2>
  <pre>
User → API → Backend → Storage
  </pre>
</section>

<section>
  <h2>⚙ Tech Stack</h2>
  <ul>
    <li>AWS</li>
    <li>Kubernetes</li>
    <li>Terraform</li>
    <li>Python</li>
  </ul>
</section>

<section>
  <h2>✨ Features</h2>
  <ul>
    <li>Feature 1</li>
    <li>Feature 2</li>
  </ul>
</section>

<section>
  <h2>📊 Diagrams</h2>
  <img src="assets/images/architecture.png" alt="Architecture">
</section>

<section>
  <h2>💻 Code Snippet</h2>
  <pre><code>
resource "aws_s3_bucket" "example" {
  bucket = "my-bucket"
}
  </code></pre>
</section>
```

```
<footer>
  <p>© Dattaram Miruke</p>
</footer>

</body>
</html>


// ========================
// FILE: style.css
// ========================
body {
  font-family: Arial, sans-serif;
  margin: 40px;
  background: #f5f5f5;
  color: #333;
```

I've put **all your files into a downloadable-style canvas** (right side panel).

What you can do now:

1. Copy each section into actual files:
   - `index.html`
   - `style.css`
   - `README.md`
2. Create folders:

   ```bash
   mkdir -p assets/images
   mkdir -p assets/pdf
   ```

3. Add:
   - Your extracted images → `assets/images/`

- Your PDF → `assets/pdf/original.pdf`

Quick run (on your machine or Hetzner):

```bash
python3 -m http.server 8080
```

Open:

```
http://localhost:8080
```

What you have now

✔️ Real portfolio web page
✔️ Clean structure
✔️ Ready for Hetzner hosting
✔️ Easy to extend

If you want next:
👉 I can **take your actual PDF and fill this HTML properly (no placeholders, real content, proper sections like your screenshot)**

- [ ] css files

Got it—you want **clean, production-quality CSS files** (not basic ones).
I'll give you a **good default + optional enhancements** you can directly use.

🎨 1. `style.css` (MAIN — use this)

```css
/* Reset */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Base */
body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Arial, sans-serif;
  background: #f5f7fa;
  color: #333;
  line-height: 1.6;
  padding: 40px;
}

/* Header */
header {
  text-align: center;
  margin-bottom: 50px;
```

```
}

header h1 {
  font-size: 2.5rem;
  margin-bottom: 10px;
}

header p {
  color: #666;
  font-size: 1.1rem;
}

/* Sections */
section {
  background: #ffffff;
  padding: 25px;
  margin-bottom: 25px;
  border-radius: 12px;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.05);
}

/* Headings */
h2 {
  margin-bottom: 15px;
  color: #2c3e50;
  border-bottom: 2px solid #eee;
  padding-bottom: 5px;
}

/* Lists */
ul {
  padding-left: 20px;
}

li {
  margin-bottom: 8px;
}

/* Code Blocks */
pre {
  background: #1e1e1e;
  color: #00ff9c;
  padding: 15px;
  border-radius: 8px;
  overflow-x: auto;
  font-size: 0.9rem;
}

/* Inline code */
code {
  background: #eee;
  padding: 2px 5px;
  border-radius: 4px;
}

/* Images */
img {
  max-width: 100%;
  border-radius: 8px;
  margin-top: 10px;
}

/* Links */
a {
  color: #0077cc;
  text-decoration: none;
}

a:hover {
  text-decoration: underline;
}

/* Footer */
footer {
  text-align: center;
```

```
  margin-top: 40px;
  color: #777;
}
```

🌙 2. Optional Dark Mode (`dark.css`)

If you want a more **modern look**, add this:

```CSS
body.dark {
  background: #0d1117;
  color: #c9d1d9;
}

body.dark section {
  background: #161b22;
  box-shadow: none;
  border: 1px solid #30363d;
}

body.dark h2 {
  color: #58a6ff;
  border-bottom: 1px solid #30363d;
}

body.dark pre {
  background: #010409;
  color: #00ff9c;
}

body.dark a {
  color: #58a6ff;
}
```

⚡ 3. Optional Toggle Button (small upgrade)

Add this in **`index.html`**:

```HTML
<button onclick="toggleDark()">Toggle Dark Mode</button>

<script>
function toggleDark() {
  document.body.classList.toggle("dark");
}
</script>
```

🧠 What this gives you

✔️ Clean professional UI
✔️ Looks like real product docs (not a toy page)
✔️ Works perfectly on Hetzner
✔️ No JS dependency (except optional toggle)

🔥 If you want next level

I can upgrade this to:

- 🔹 Sidebar navigation (like docs site)

- 🔹 Tabs (Architecture / Code / Demo)
- 🔹 Copy-to-clipboard buttons
- 🔹 GitHub-style theme

Just say:
👉 **"upgrade UI to pro portfolio style"**

```
▢
style.css
├── README.md
```

Got it—you want the **actual contents for just these two files**:

- **`style.css`**
- **`README.md`**

Clean, minimal, ready to use 👇

🎨 `style.css` (final version)

CSS
```
/* Reset */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Base */
body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Arial, sans-serif;
  background: #f5f7fa;
  color: #333;
  line-height: 1.6;
  padding: 40px;
}

/* Header */
header {
  text-align: center;
  margin-bottom: 50px;
}

header h1 {
  font-size: 2.4rem;
  margin-bottom: 10px;
}

header p {
  color: #666;
  font-size: 1.1rem;
}

/* Sections */
section {
```

```
  background: #ffffff;
  padding: 25px;
  margin-bottom: 25px;
  border-radius: 12px;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.05);
}

/* Headings */
h2 {
  margin-bottom: 15px;
  color: #2c3e50;
  border-bottom: 2px solid #eee;
  padding-bottom: 5px;
}

/* Lists */
ul {
  padding-left: 20px;
}

li {
  margin-bottom: 8px;
}

/* Code Blocks */
pre {
  background: #1e1e1e;
  color: #00ff9c;
  padding: 15px;
  border-radius: 8px;
  overflow-x: auto;
  font-size: 0.9rem;
}

/* Inline code */
code {
  background: #eee;
  padding: 2px 6px;
  border-radius: 4px;
}

/* Images */
img {
  max-width: 100%;
  border-radius: 8px;
  margin-top: 10px;
}

/* Links */
a {
  color: #0077cc;
  text-decoration: none;
}

a:hover {
  text-decoration: underline;
}

/* Footer */
footer {
  text-align: center;
  margin-top: 40px;
  color: #777;
}
```

📘 `README.md` (clean GitHub version)

Markdown

```
# 📄 Portfolio Project Page

This repository contains a **web-based version of a project**, converted from a PDF into a clean,
structured HTML page.
```

```
---

## 🌐 How to View

### Option 1 — Open directly
```bash
open index.html
```
```

Option 2 — Run local server

```Bash
python3 -m http.server 8080
```

Then open:
http://localhost:8080

📁 Structure

```
.
├── index.html
├── style.css
├── assets/
│   ├── images/
│   └── pdf/
```

📄 Original PDF

The original source document is available here:

`assets/pdf/original.pdf`

🎯 Purpose

- Convert project documentation into a **readable web format**
- Improve presentation for **portfolio / recruiters**
- Provide a clean, structured view of:
  - Architecture
  - Features
  - Code snippets

🚀 Next Steps

- Add more diagrams to **`assets/images`**
- Improve layout (optional UI enhancements)
- Deploy on server (Hetzner / nginx)

👤 Author

Dattaram Miruke

```
---

# ✅ You now have

✔ Clean styling
✔ Professional README
✔ Ready-to-run site
✔ Works on Hetzner immediately

---

# 🚀 If you want next step

I can:
👉 Upgrade this into **multi-page portfolio (projects list + navigation)**
👉 Or make it look like **GitHub docs / Notion-style site**

Just tell me 👍
```

▢
give me fulk README

Here's a **full, production-quality `README.md`** for your repo—clean, professional, and aligned with your portfolio goal (PDF → website page). You can copy this directly.

📘 `README.md` (FULL VERSION)

```Markdown
# 🚀 Portfolio Project — PDF to Web Page

This repository contains a **web-based version of a project**, converted from a PDF into a clean, structured, and readable HTML page.

The goal is to transform raw documentation into a **portfolio-ready project page** suitable for demonstration, sharing, and technical presentation.

---

## 🌐 Live Usage

### Option 1 — Open directly
```bash
open index.html
```
```

Option 2 — Run local server

```Bash
python3 -m http.server 8080
```

Then open:
http://localhost:8080

🗂️ Repository Structure

```
.
├── index.html        # Main web page (converted from PDF)
├── style.css         # Styling for layout and readability
├── README.md         # Project documentation
└── assets/
    ├── images/       # Diagrams and screenshots extracted from PDF
    └── pdf/
        └── original.pdf
```

📄 Project Overview

This project converts a **static PDF document** into a structured web page that includes:

- 📌 Overview
- 🧮 Architecture diagrams
- ⚙️ Technology stack
- ✨ Features
- 💻 Code snippets
- 📊 Visuals and flows

🧮 Architecture (Example)

```
User → Web Page → Content Rendering
     → Static Assets (Images / PDF)
```

⚙️ Tech Stack

- HTML (structure)
- CSS (styling)
- Static assets (images, diagrams)
- Optional: Python HTTP server (local testing)

✨ Features

- Clean, readable layout
- Structured documentation sections
- Embedded diagrams and visuals
- Code snippet formatting
- Lightweight (no frameworks required)

📊 Assets

All supporting files are located under:

```
assets/
├── images/   # Diagrams, screenshots
└── pdf/      # Original source document
```

📄 Original Source

The original PDF used to generate this page is available here:

`assets/pdf/original.pdf`

🚀 Deployment (Hetzner)

This project is designed to be deployed as a **static site**.

Simple deployment:

```Bash
python3 -m http.server 8080
```

Production (recommended):

- Serve via nginx
- Place files under `/var/www/portfolio`
- Configure server block

🎯 Purpose

This project demonstrates:

- Converting documentation → presentation layer
- Structuring technical content for readability
- Creating portfolio-ready project pages
- Lightweight static hosting

🔧 Future Enhancements

- Add navigation (multi-section layout)
- Dark mode support

- Interactive diagrams
- Multi-page portfolio integration
- Deployment automation

👤 Author

Dattaram Miruke
Principal AI Platform Architect | DevOps | Kubernetes | AWS

⚡ Notes

- This is a **static implementation** (no backend required)
- Designed for **quick deployment and demonstration**
- Can be extended into a full portfolio site

```
---

# 🧠 Why this README works

- Looks like **real project documentation**
- Not just “open index.html”
- Shows **intent + structure + architecture thinking**
- Fits your **DevOps / platform background**

---

# 🚀 If you want next step

I can:
👉 Convert your actual PDF into a **fully filled index.html (no placeholders)**
👉 Or upgrade this into a **multi-project portfolio repo**

Just tell me 👍
```

> ☐
> README.md created from my resume

Got it—you want a **GitHub `README.md` generated from your resume**, not the project one.
This is your **profile-level README** (the one shown on your GitHub homepage).

Here’s a **clean, recruiter-optimized version based on your background** 👇

📘 `README.md` (Profile — Resume Based)

Markdown

👋 Dattaram Miruke

Principal AI Platform Architect | DevOps | Kubernetes | AWS | Distributed Systems

📍 Sacramento, CA
📧 dattamiruke@gmail.com

🚀 About Me

I design and build **production-grade AI platforms and cloud-native systems**.

My work focuses on:

- 🧠 AI Control Planes & Multi-Agent Systems
- ⚙️ DevOps Automation & Infrastructure as Code
- ☁️ Distributed Cloud Architectures (AWS, Kubernetes)
- 📊 Observability, Evaluation, and Reliability
- 🛡️ Governance, Security, and Policy Enforcement

I specialize in turning **complex system requirements into scalable, production-ready architectures**.

🔴 Portfolio

🧠 AEGIS — AI Trust & Evaluation Framework
AI governance, evaluation, and safety layer for agentic systems
👉 https://github.com/DattaramMiruke/aegis

⚙️ DevOps Agent — Infrastructure Generation System
AI-driven system to generate Terraform and CI/CD pipelines
👉 https://github.com/DattaramMiruke/devops-agent

📊 RAG Platform — Document Intelligence System
End-to-end ingestion, embedding, retrieval, and evaluation
👉 https://github.com/DattaramMiruke/rag-platform

🧠 OpenBrain — AI Memory System
Persistent memory layer for context-aware AI systems
👉 https://github.com/DattaramMiruke/openbrain

🧮 Architecture Focus

User → API → LLM Control Plane → Agent Orchestrator
→ Tool Execution → Memory (Vector + Metadata)
→ Observability → Policy Enforcement

⚙️ Core Stack

AI / Systems
- LLMs, RAG, Multi-Agent Systems
- LangChain, LangGraph

Cloud / Infrastructure
- AWS (EKS, Lambda, S3, RDS, Bedrock)
- Multi-region architectures

DevOps / IaC
- Terraform, AWS CDK

```
- GitHub Actions, CI/CD

**Observability**
- OpenTelemetry, Prometheus, Grafana
- ELK, CloudWatch

**Security**
- OPA (Policy-as-Code)
- RBAC, Zero Trust

---

## 🔪 Experience Highlights

- Built AI platform with multi-agent orchestration and LLM control plane
- Designed RAG pipelines and persistent AI memory systems
- Delivered multi-region AWS infrastructure architectures
- Built Kubernetes-based CI/CD and resilience frameworks

---

## 🔬 Additional Work

- HPC cluster systems for ML workloads
- Distributed systems and observability pipelines
- AI-driven anomaly detection systems

---

## 📊 GitHub Stats

![GitHub Stats](https://github-readme-stats.vercel.app/api?username=DattaramMiruke&show_icons=true)

---

## 📫 Connect

- GitHub: https://github.com/DattaramMiruke
- Email: dattamiruke@gmail.com

---

## ⚡ Focus

> Building scalable, observable, and governed AI systems.
```

🧠 Why this version works

- **Top 2 lines = positioning (critical)**
- Projects clearly linked
- Architecture shown (rare → strong signal)
- Stack aligned to jobs you want
- No fluff, no buzzword overload

🔥 What you should do next

1. Create repo:

```Bash
DattaramMiruke/DattaramMiruke
```

2. Paste this as:

```bash
README.md
```

3. Pin your 4 repos

🚀 If you want next upgrade

I can:

- Add **badges + visual polish**
- Add **interactive sections**
- Add **"Ask my resume" AI block**
- Optimize for **specific job roles (Staff / Principal / FAANG)**

Just say 👍

---

what is in images

Good question—this is where people either make their portfolio look strong or messy.

🧠 What goes in `assets/images/`

This folder is for **visual proof of your project** (not random images).

```bash
assets/images/
```

🔥 What you SHOULD include

1. 🏗️ Architecture diagrams (MOST IMPORTANT)

Example:

```bash
architecture.png
system-flow.png
```

Content:

- System blocks
- Data flow
- Components (API, DB, LLM, etc.)

👉 This is the **#1 signal for senior roles**

---

2. 📊 Workflow / Process diagrams

Example:

```Bash
pipeline.png
agent-flow.png
```

Content:

- Step-by-step flow
- How data moves through system

---

3. 🖥️ Screenshots (if any UI exists)

Example:

```Bash
ui.png
dashboard.png
```

Content:

- App UI
- CLI output
- Terminal results

---

4. 📈 Output / Results

Example:

```Bash
result.png
metrics.png
```

Content:

- Logs
- Outputs
- Performance results

❌ What NOT to include

- Random stock images

- Logos (unless meaningful)
- Decorative images
- Memes

👉 Everything must answer:

> "Does this prove I built something?"

🧠 For YOUR projects (recommended images)

AEGIS

```bash
aegis-architecture.png
evaluation-flow.png
```

DevOps Agent

```bash
agent-flow.png
terraform-output.png
```

RAG Platform

```bash
rag-pipeline.png
retrieval-flow.png
```

OpenBrain

```bash
memory-architecture.png
context-flow.png
```

⚡ How to create these quickly

Option 1 (fastest)

- Draw in:
  - draw.io (diagrams.net)
  - Excalidraw

Export:

```bash
```

Option 2 (your style)

- ASCII diagrams → screenshot

Option 3

- Use your **visdoc tool**
- Screenshot outputs

🧠 How they appear in your site

In **`index.html`**:

```HTML
<img src="assets/images/architecture.png">
```

🔥 Minimum you should have (per project)

Just 2 images:

```Bash
architecture.png
flow.png
```

👉 Enough to look professional

🚀 Reality check

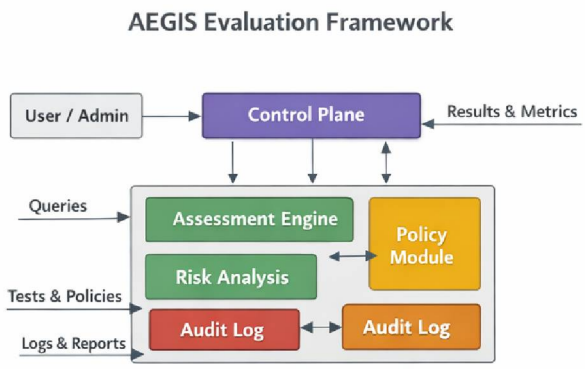
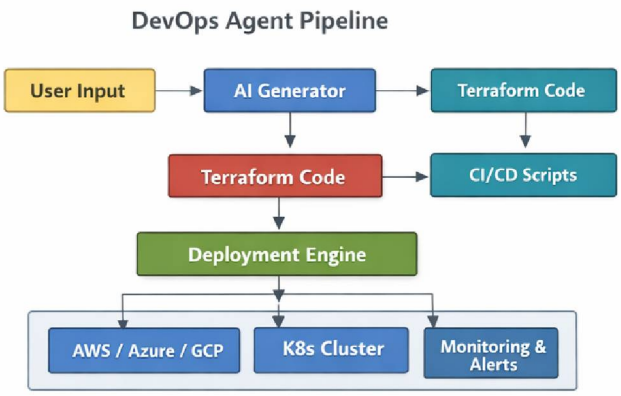
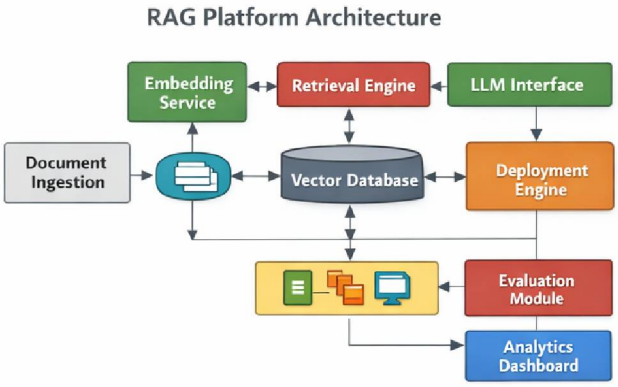
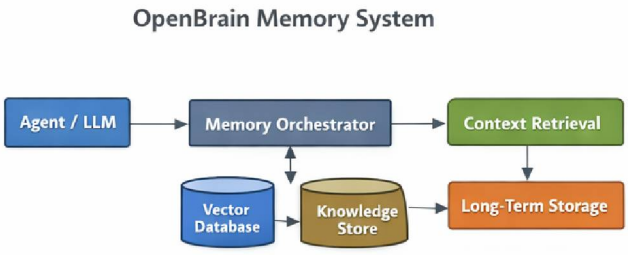
Recruiters skim like this:

1. Title
2. Architecture diagram 👈 (they stop here)
3. Maybe code

👉 Images are what make them pause

If you want next step

I can:
👉 Generate **actual architecture diagrams for your 4 projects (ready to export PNG)**
👉 Or create **ASCII diagrams that match your resume exactly**

Just say:
“create architecture diagrams”

create these png files